

FLASK MINI PROJECT REPORT

Student Management System

Student Name: Bammidi Rakesh
Roll Number: 23981A04Q9
Department: Electronics and Communication Engineering (ECE)
College: Raghu Engineering College
Project: Task 4 – Flask Mini Project

1. Objective

Develop a web-based Student Management System using Flask, Jinja2 templates, SQLite, and Bootstrap. The application demonstrates routing, GET and POST form handling, and complete CRUD operations.

2. Requirements Mapping

- Flask routing and templates
- GET and POST form handling
- Basic CRUD and SQLite data storage
- Clean Bootstrap UI
- GitHub-ready project structure

3. Technologies Used

Python, Flask, SQLite, HTML5, Jinja2 templating, Bootstrap 5.3.3, and VS Code.

4. Application Features

Add Student (Create), View Students (Read), Edit Student (Update), Delete Student (Delete), and a responsive Bootstrap interface.

6. Flask Routes

/ – displays all students. /add – GET displays the form and POST inserts a student. /edit/<student_id> – GET displays existing data and POST updates it. /delete/<student_id> – deletes a student and redirects to the list.

7. CRUD Flow

Create → POST → SQLite INSERT → list; Read → GET → SQLite SELECT → table; Update → POST → SQLite UPDATE → list; Delete → confirmation → SQLite DELETE → list.

5. Database Design

Column	Type	Constraint	Purpose
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique student ID
name	TEXT	NOT NULL	Student name
email	TEXT	NOT NULL	Email address
department	TEXT	NOT NULL	Department

8. Testing and Screenshots

Screenshot 1: Main student list showing records and CRUD action buttons.

How to Submit Your Tasks

1. For each task:

- Create a separate document (DOC, DOCX, or PDF) for **eac**
- Add screenshots, code snippets, hosted links, and GitHub

2. Upload your documents:

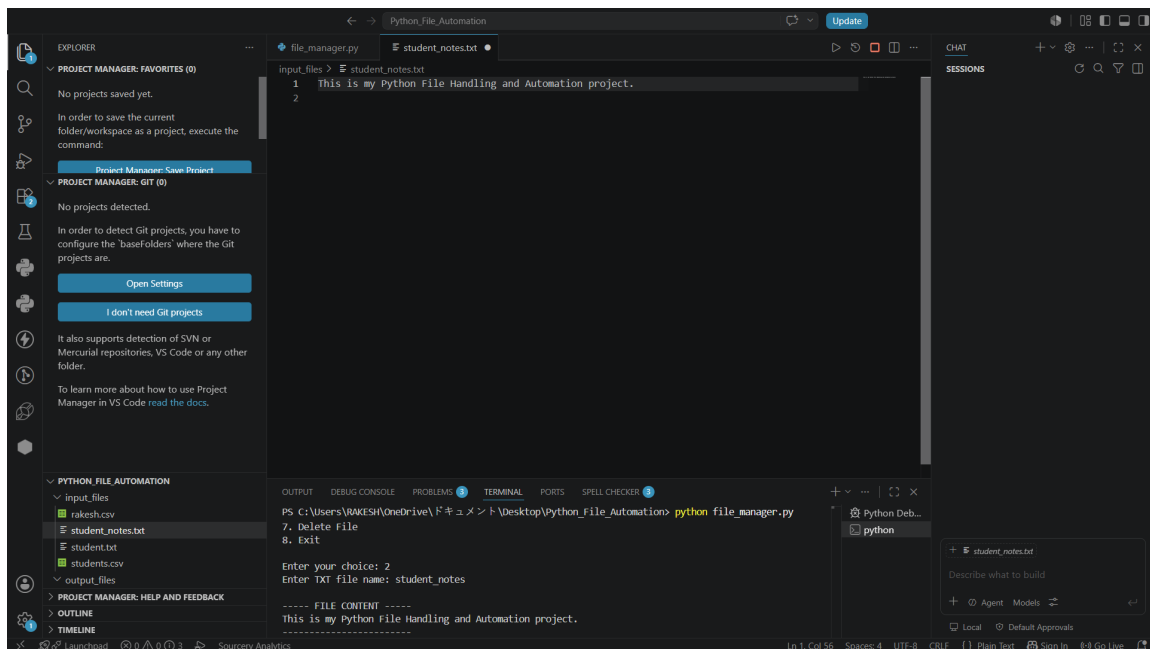
- Upload to **Google Drive** or **GitHub** (public link).
- Ensure anyone can view the files.

3. Submit your links:

- Copy the public links for each task document.
- Go to the [Task Submission](#) page.
- Paste links clearly with task numbers.

Tip: Well-organized documentation = faster review!

Screenshot 2: Add Student form demonstrating GET/POST form handling.



Screenshot 3: Edit Student form with existing values ready for update.

How to Submit Your Tasks

1. For each task:

- Create a separate document (DOC, DOCX, or PDF) for **eac**
- Add screenshots, code snippets, hosted links, and GitHub

2. Upload your documents:

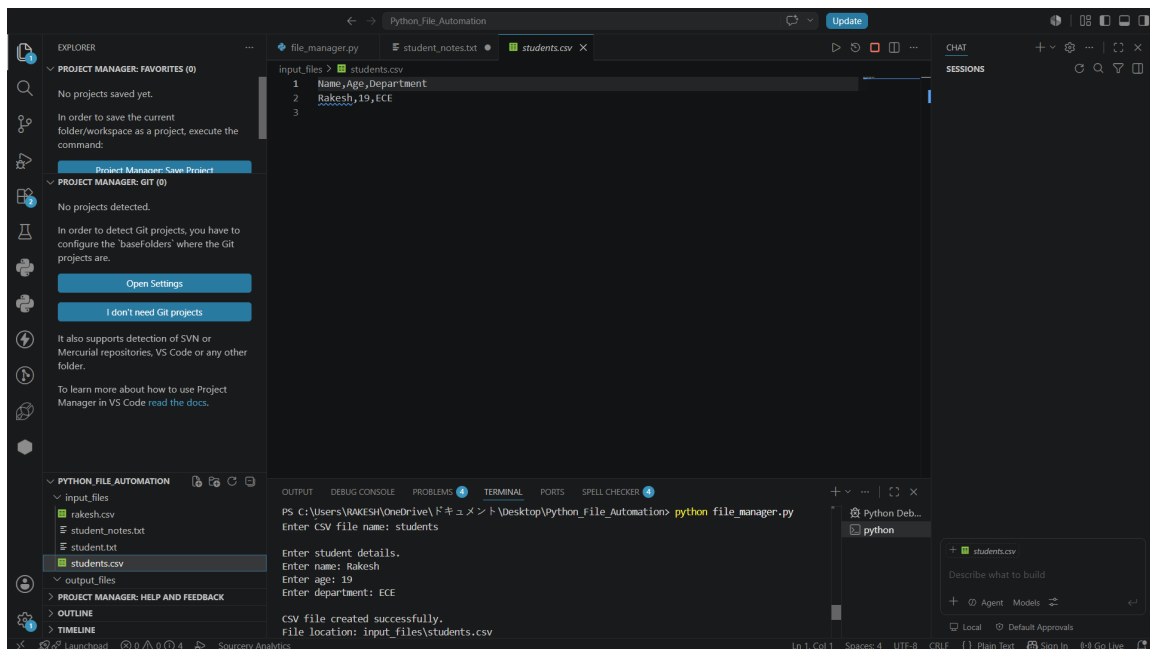
- Upload to **Google Drive** or **GitHub** (public link).
- Ensure anyone can view the files.

3. Submit your links:

- Copy the public links for each task document.
- Go to the [Task Submission](#) page.
- Paste links clearly with task numbers.

Tip: Well-organized documentation = faster review!

Screenshot 4: Delete confirmation demonstrating the delete workflow.



9. Source Code – app.py

```
from flask import Flask, render_template, request, redirect, url_for
import sqlite3

app = Flask(__name__)
DATABASE = "students.db"

def get_db_connection():
    connection = sqlite3.connect(DATABASE)
    connection.row_factory = sqlite3.Row
    return connection

def create_table():
    connection = get_db_connection()
    connection.execute("""
    CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    department TEXT NOT NULL
    )
    """)
    connection.commit()
    connection.close()

@app.route("/")
def index():
    connection = get_db_connection()
    students = connection.execute(
    "SELECT * FROM students ORDER BY id DESC"
    ).fetchall()
    connection.close()
    return render_template("index.html", students=students)

@app.route("/add", methods=["GET", "POST"])
def add_student():
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        department = request.form["department"]
        connection = get_db_connection()
        connection.execute(
        "INSERT INTO students (name, email, department) VALUES (?, ?, ?)",
        (name, email, department)
        )
        connection.commit()
        connection.close()
        return redirect(url_for("index"))
    return render_template("add_student.html")

@app.route("/edit/<int:student_id>", methods=["GET", "POST"])
def edit_student(student_id):
    connection = get_db_connection()
    student = connection.execute(
    "SELECT * FROM students WHERE id = ?", (student_id,)
    ).fetchone()
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        department = request.form["department"]
        connection.execute(
        "UPDATE students SET name = ?, email = ?, department = ? WHERE id = ?",
        (name, email, department, student_id)
        )
        connection.commit()
        connection.close()
        return redirect(url_for("index"))
    connection.close()
    return render_template("edit_student.html", student=student)

@app.route("/delete/<int:student_id>")
def delete_student(student_id):
    connection = get_db_connection()
    connection.execute("DELETE FROM students WHERE id = ?", (student_id,))
    connection.commit()
    connection.close()
    return redirect(url_for("index"))

if __name__ == "__main__":
    create_table()
    app.run(debug=True)
```

10. Conclusion

The project successfully demonstrates Flask routing, Jinja2 templates, GET and POST requests, SQLite persistence, and complete CRUD functionality with a clean Bootstrap interface.

11. Submission

GitHub repository: <https://github.com/bammidirakesh/Flask-Student-Management-System>